

Построение пайплайнов данных

Декларативная оптимизация

Дмитрий Малахов



Максим Новосельцев

27 февраля 2026 года

RDD vs DataFrame — эволюция API

- Появление Spark RDD показало, что обработка больших данных может быть серьезно оптимизирована, однако она все еще требовала большой квалификации и в ряде случаев выглядела громоздко.
- Spark DataFrame появился для расширения потенциального круга пользователей, скрытой оптимизации вычислений, их упрощения и сокращения их определения.
- Потеряв в гибкости и статической типизации, Spark DataFrame инкапсулировал достаточно много оптимизаций. В ряде задач это играет ключевую роль, минимальным количеством кода можно добиться производительности, которая достигается в Spark RDD большими усилиями. Однако в предобработке данных гибкость Spark RDD может существенно упростить вычисления и ускорить их, а строгая типизация способствует меньшему количеству ошибок.
- **Главное отличие:** DataFrame — это **декларативный API**. Вы говорите **ЧТО** сделать, а не **КАК**. Spark сам выбирает план вычислений (**не всегда оптимально**).

Catalyst Optimizer — сердце оптимизации

Ваш код (`df.filter(...).join(...).groupBy(...)`)

↓

1. Анализ (Analysis) – проверка типов, колонок, разрешение ссылок

↓

2. Логическая оптимизация (Logical Optimization)

- Projection pushdown – читай только нужные колонки
- Predicate pushdown – фильтруй как можно раньше
- Constant folding – вычисляй константы заранее

↓

3. Физическое планирование (Physical Planning)

- Выбор стратегии join (`BroadcastHashJoin` vs `SortMergeJoin`)
- Выбор алгоритмов агрегации

↓

4. Генерация кода (Code Generation) – Tungsten

- Генерация байт-кода Java
- Работа с памятью off-heap

Tungsten — эффективное управление памятью

Три кита Tungsten:

1. Off-heap память (Binary Format)

- Данные хранятся не как объекты, а в бинарном формате
- Меньше GC, меньше накладных расходов

2. Cache-aware вычисления

- Оптимизация под иерархию кэша процессора
- Минимизация cache misses

3. Whole-stage code generation

- Генерация оптимизированного байт-кода на этапе компиляции запроса
- Устранение виртуальных вызовов функций

Сложные типы в Spark

- JSON/XML данные часто вложенные
- Избегаем join'ов — храним связанные данные вместе
- Более естественное представление

Тип	Пример данных	Spark тип
Struct	<code>{"name": "John", "age": 30}</code>	<code>StructType</code>
Array	<code>[1, 2, 3, 4, 5]</code>	<code>ArrayType</code>
Map	<code>{"RU": "Москва", "US": "Вашингтон"}</code>	<code>MapType</code>

Работа со Struct (вложенные данные)

```
{
  "user": {
    "name": "Иван",
    "contacts": {
      "email": "ivan@mail.ru",
      "phone": "+7-123-456-78-90"
    }
  },
  "purchase": {
    "amount": 1500,
    "items": ["книга", "ручка"]
  }
}
```

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

df = spark.read.json("data.json")

# Доступ к полям структуры
df.select(
    col("user.name"),
    col("user.contacts.email"),
    col("purchase.amount")
).show()

# Создание структуры на лету
from pyspark.sql.functions import struct
df.withColumn("user_info",
    struct(
        col("user.name").alias("name"),
        col("user.contacts.email").alias("email")
    )
)
```

Работа с Array

```
from pyspark.sql.functions import array, split

df = spark.createDataFrame([(1, "a,b,c"), (2, "d,e")], ["id", "csv"])
df.withColumn("tags", split(col("csv"), ","))
```

Функция

Описание

`array_contains(col, value)`

Проверяет наличие элемента

`explode(col)`

Разворачивает массив в строки

`size(col)`

Длина массива

`array_distinct(col)`

Уникальные элементы

`array_intersect(col1, col2)`

Пересечение массивов

explode — разворачивание массивов

Исходно:

id	items
1	[a, b, c]
2	[d, e]

После explode:

id	items	item
1	[a,b,c]	a
1	[a,b,c]	b
1	[a,b,c]	c
2	[d,e]	d
2	[d,e]	e

```
from pyspark.sql.functions import explode, collect_list, collect_set

df_exploded = df.withColumn("item", explode("items")).drop("items")

df_both = df_exploded.groupBy("id").agg(
    collect_list("item").alias("items_list"),
    collect_set("item").alias("items_set")
)
```


Работа с Map

```
from pyspark.sql.functions import create_map, lit, col, map_keys, map_values, element_at

# Исходные данные
df = spark.createDataFrame([(1, "RU", "Moscow")], ["id", "code", "city"])

# Создание map колонки
df_map = df.withColumn("info", create_map(lit("code"), col("code"), lit("cap"), col("city")))

# Работа с map: ключи, значения, доступ по ключу
df_result = df_map.select(
    "id",
    map_keys("info").alias("keys"),      # все ключи map
    map_values("info").alias("vals"),    # все значения map
    element_at("info", "code").alias("code") # значение по ключу "code"
)
```

Оконные функции — зачем они нужны

```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum, col, round

df = spark.createDataFrame([
    (1,100), (1,200), (1,300), (2,150), (2,350)
], ["user_id", "amount"])

# Проблема: groupBy теряет детали
df.groupBy("user_id").agg(sum("amount").alias("total")).show()

# Решение: оконные функции сохраняют строки
window_spec = Window.partitionBy("user_id")
result = df.withColumn("user_total", sum("amount").over(window_spec)) \
    .withColumn("share", round(col("amount")/col("user_total")*100,2))
```

Синтаксис оконных функций

```
from pyspark.sql.window import Window
```

Три компонента оконной функции:

```
window_spec = Window \  
    .partitionBy("user_id") \           # 1. Группировка  
    .orderBy("timestamp") \           # 2. Сортировка  
    .rowsBetween(Window.unboundedPreceding, Window.currentRow) # 3. Рамки
```

Варианты рамок:

```
# rowsBetween(-∞, current) - от начала до текущей  
# rowsBetween(current, +∞) - от текущей до конца  
# rowsBetween(-1, 1)      - предыдущая, текущая, следующая  
# rangeBetween(-100, 0)   - по значению (последние 100 сек)
```

Константы:

```
# Window.unboundedPreceding = -∞  
# Window.unboundedFollowing = +∞  
# Window.currentRow         = текущая строка
```

Агрегатные оконные функции

```
from pyspark.sql.functions import sum, avg, min, max, count

df.withColumn("running_total", sum("amount").over(window_spec))
df.withColumn("avg_amount", avg("amount").over(window_spec))
```

Ранжирующие оконные функции

```
from pyspark.sql.functions import row_number, rank, dense_rank, ntile

# Номер строки в окне (1,2,3,4,...)
df.withColumn("row_num", row_number().over(window_spec))

# Ранг с пропусками (1,1,3,4,...)
df.withColumn("rank", rank().over(window_spec))

# Ранг без пропусков (1,1,2,3,...)
df.withColumn("dense_rank", dense_rank().over(window_spec))

# Разбиение на N групп
df.withColumn("bucket", ntile(10).over(window_spec))
```

Оконные функции смещения

```
from pyspark.sql.functions import lag, lead, first, last

# Предыдущее значение
df.withColumn("prev_amount", lag("amount", 1).over(window_spec))

# Следующее значение
df.withColumn("next_amount", lead("amount", 1).over(window_spec))

# Первое значение в окне
df.withColumn("first_amount", first("amount").over(window_spec))
```

Кейс: Скользящее среднее (Moving Average)

```
from pyspark.sql.window import Window
from pyspark.sql.functions import avg, col

# Окно: последние 3 транзакции по пользователю
window_spec = Window.partitionBy("user_id") \
    .orderBy("timestamp") \
    .rowsBetween(-2, Window.currentRow)

# Скользящее среднее за 3 последние покупки
df.withColumn("moving_avg_3", avg("amount").over(window_spec)).show()

# Результат:
# user_id | amount | timestamp | moving_avg_3
# 1       | 100    | 10:00     | 100.0
# 1       | 200    | 10:05     | 150.0
# 1       | 300    | 10:10     | 200.0
# 1       | 400    | 10:15     | 300.0 # avg(200,300,400)
```

Кейс: Топ-N записей в группе

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number

# Окно: по пользователю, сортировка по сумме (от большей к меньшей)
window_spec = Window.partitionBy("user_id") \
    .orderBy(col("amount").desc())

# Добавляем номер строки в окне
df_with_rank = df.withColumn("rn", row_number().over(window_spec))

# Оставляем только топ-3
top3_per_user = df_with_rank.filter(col("rn") <= 3)

# Альтернатива с rank() – если нужны все с одинаковой суммой
from pyspark.sql.functions import rank
df_with_rank = df.withColumn("rank", rank().over(window_spec))
top_with_duplicates = df_with_rank.filter(col("rank") <= 3)
```


Кейс: Разница с предыдущей записью

```
from pyspark.sql.window import Window
from pyspark.sql.functions import lag, col, round

# Задача: для каждого пользователя посчитать изменение суммы
# покупки по сравнению с предыдущей транзакцией

window_spec = Window.partitionBy("user_id").orderBy("timestamp")

result = df.withColumn("prev_amount", lag("amount", 1).over(window_spec)) \
    .withColumn("amount_diff", col("amount") - col("prev_amount")) \
    .withColumn("percent_change",
                round((col("amount") - col("prev_amount")) /
                      col("prev_amount") * 100, 2))
```

UDF — мощь и опасность

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

# Определяем обычную Python-функцию
def category_name(category_id):
    mapping = {1: "Электроника", 2: "Одежда", 3: "Книги"}
    return mapping.get(category_id, "Другое")

# Регистрируем как UDF
category_name_udf = udf(category_name, StringType())

# Используем
df.withColumn("category_name", category_name_udf(col("category_id")))
```

Цена использования UDF

1. Разрыв оптимизации Catalyst

- Spark не знает, что делает UDF
- Нет кодогенерации
- Черный ящик для оптимизатора

2. Сериализация

- Данные конвертируются из внутреннего формата Tungsten в объекты
- Огромные накладные расходы

Альтернативы UDF

1. Встроенные функции

```
from pyspark.sql.functions import when, col
df.withColumn("category_name",
    when(col("category_id") == 1, "Электроника")
    .when(col("category_id") == 2, "Одежда")
    .otherwise("Другое"))
```

2. Функции для массивов

```
from pyspark.sql.functions import transform
df.withColumn("prices_with_tax", transform(col("prices"), lambda x: x * 1.2))
```

3. Pandas UDF (векторизованные)

```
from pyspark.sql.functions import pandas_udf
from pyspark.sql.types import StringType
import pandas as pd
@pandas_udf(StringType())
def category_name_pd(category_ids: pd.Series) -> pd.Series:
    mapping = {1: "Электроника", 2: "Одежда"}
    return category_ids.map(mapping).fillna("Другое")
```

Когда UDF действительно нужны

1. Сложная бизнес-логика

```
def calculate_risk_score(user_data):  
    # 100 строк логики с условиями, циклами, вызовами API  
    return risk_score  
risk_udf = udf(calculate_risk_score, DoubleType())
```

2. Работа с внешними библиотеками

```
import nltk  
def sentiment_analysis(text):  
    return nltk.sentiment.SentimentIntensityAnalyzer().polarity_scores(text)  
sentiment_udf = udf(sentiment_analysis, StringType())
```

3. Обработка сложных форматов

```
import hashlib, json  
def compute_hash(struct_data):  
    return hashlib.sha256(json.dumps(struct_data).encode()).hexdigest()
```

Правило: Если можно встроенными — используй их. UDF — крайняя мера

Профилирование DataFrame

```
# Обычный explain (физический план)
df.explain()

# Вывод: только физический план выполнения

# Расширенный explain (логический + физический план)
df.explain(True)

# Вывод:
# == Parsed Logical Plan == (сырой SQL запрос)
# == Analyzed Logical Plan == (после проверки типов и колонок)
# == Optimized Logical Plan == (после оптимизаций Catalyst)
# == Physical Plan == (как будет выполняться на кластере)

# Пример для join с broadcast
from pyspark.sql.functions import broadcast
df.join(broadcast(small_df), "key").explain()

# В physical plan будем: BroadcastHashJoin
```

Итоги

1. Если задача хорошо ложится на DataFrame, сначала используем его.
2. К RDD спускаемся там, где входные данные не в табличном виде, и там где требуется генерация разных выходных форматов в рамках одного прохода по входным данным.
3. Изучаем встроенные функции — их >200, вероятно, нужная уже есть.
4. Если вместо UDF есть возможность использовать Pandas UDF — используем его.
5. Храним вложенные данные как struct/array — меньше join'ов (для небольших иерархий).
6. Оконные функции являются отличной альтернативой агрегатным.
7. Проверяем план — df.explain() покажет, работает ли оптимизация
8. Кешируем только то, что используется и не забываем про broadcast
9. Работая с DataFrame генерить код через LLM проще и быстрее (используем как можно чаще).

Домашнее задание

Задание: Реализуйте 3 кейса, где DataFrame гарантированно быстрее любого. Для каждого кейса приведите код и объясните, какая оптимизация Catalyst/Tungsten дает выигрыш.

Возможные кейсы:

- Множественные агрегации — `df.groupBy().agg(sum, avg, min)` vs несколько RDD-действий
- Оконные функции — топ-N в группе против сортировки + фильтрации на RDD
- Работа с вложенными типами — `struct/array` без сериализации vs парсинг на RDD
- Условная логика (`when/otherwise`) против множественных `filter` + `union` на RDD

(*) Найдите 2 кейса, где SQL-запрос выполняется быстрее эквивалентного DataFrame API. Сравните планы выполнения через `explain()` и объясните причину разницы.

Результат: Отчет с таблицами производительности и выводами, когда какой API выбирать.

